

Institut Supérieure d'Informatique et de
Multimédia de Sfax



Atelier de Programmation

Chapitre 4

Les sous programmes

Enseignant : Ramzi ZOUARI

Filière : P-LSI

Année universitaire : 2025-2026

Plan

1. Introduction
2. Structure d'une fonction
3. Les appels de fonction
4. Passage des paramètres

1. Introduction

- Dans le langage C, les sous-programmes (appelés aussi fonctions) servent à découper un programme en blocs réutilisables pour éviter la redondance et améliorer la lisibilité
- Il existe deux types de fonctions:
 - **Fonctions standards ou prédéfinis**: comme printf(), scanf(), malloc(), free(), ...
 - **Fonctions définies par l'utilisateur**: créées par le programmeur pour résoudre des tâches spécifiques

```
#include<stdio.h>

// Definition (implementation) de(s) fonction(s) utilisateur

int main () {

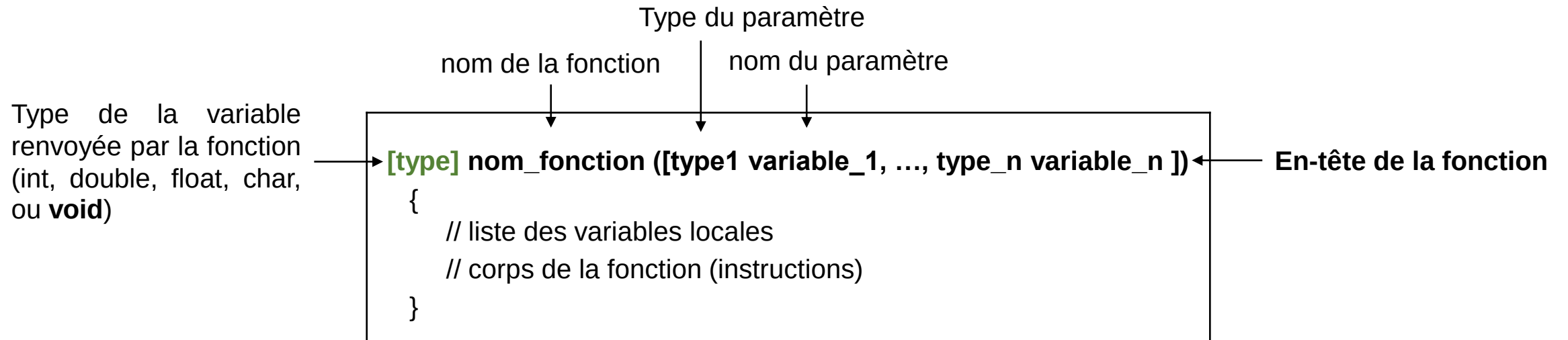
    // programme principal contenant l'appel au(x) fonction(s) utilisateur

    return 0;

}
```

2. Structure d'une fonction

- En C, une fonction doit avoir la syntaxe suivante:



Fonction avec variable renvoyée

```
int somme (int a, int b) {  
    int s;  
    s = a + b;  
    return s;  
}
```

Fonction sans variable renvoyée

```
void somme (int a, int b) {  
    int s;  
    s = a + b;  
    printf("%d + %d = %d", a, b, c);  
}
```

2. Structure d'une fonction

Exemple 1

```
#include<stdio.h>

int somme (int a, int b)
{
    int s;
    s = a + b;
    return s;
}

int main () {
    int m=2, n=3, k;
    k = somme(m , n);
    printf("%d + %d = %d", m, n, k);
    return 0;
}
```

Exemple 2

```
#include<stdio.h>

void somme (int a, int b)
{
    int s;
    s = a + b;
    printf("%d + %d = %d", a, b, s);
}

int main () {
    int m=2, n=3;
    somme(m , n);
    return 0;
}
```

2. Structure d'une fonction

Exemple 1

```
#include<stdio.h>
```

```
int somme (int a, int b)
```

```
{
```

```
    int s;
```

```
    s = a + b;
```

```
    return s;
```

```
}
```

```
int main () {
```

```
    int m=2, n=3, k;
```

```
    k = somme (m , n);
```

```
    printf("%d + %d = %d", m, n, k);
```

```
    return 0;
```

```
}
```

Les paramètres spécifiés dans la définition de la fonction sont appelés **paramètres formels**

Variables locales
connues seulement à
l'intérieur de la fonction

La durée de vie est limitée à la
durée d'exécution de la fonction
(créées lors de l'appel de la fonction
et détruites à la fin de la fonction)

Les paramètres spécifiés lors de l'appel à la fonction sont appelés **paramètres effectifs**

Variables globales accessibles
depuis n'importe où dans le
programme

La durée de vie s'étend sur toute
l'exécution du programme

3. Appels de fonction

Exemple 1

```
#include<stdio.h>
```

```
int somme (int a, int b)
```

```
{
```

```
    int s;
```

```
    s = a + b;
```

```
    return s;
```

```
}
```

```
int main () {
```

```
    int m=2, n=3, k;
```

```
    k = somme (m , n);
```

```
    printf("%d + %d = %d", m, n, k);
```

```
    return 0;
```

```
}
```

1

Les valeurs des paramètres effectifs sont copiés dans l'ordre dans les paramètres formels

- La valeur de *m* sera copiée dans *a*
- La valeur de *n* sera copiée dans *b*

2

Calcul de la somme en utilisant les valeurs des paramètres effectives

3

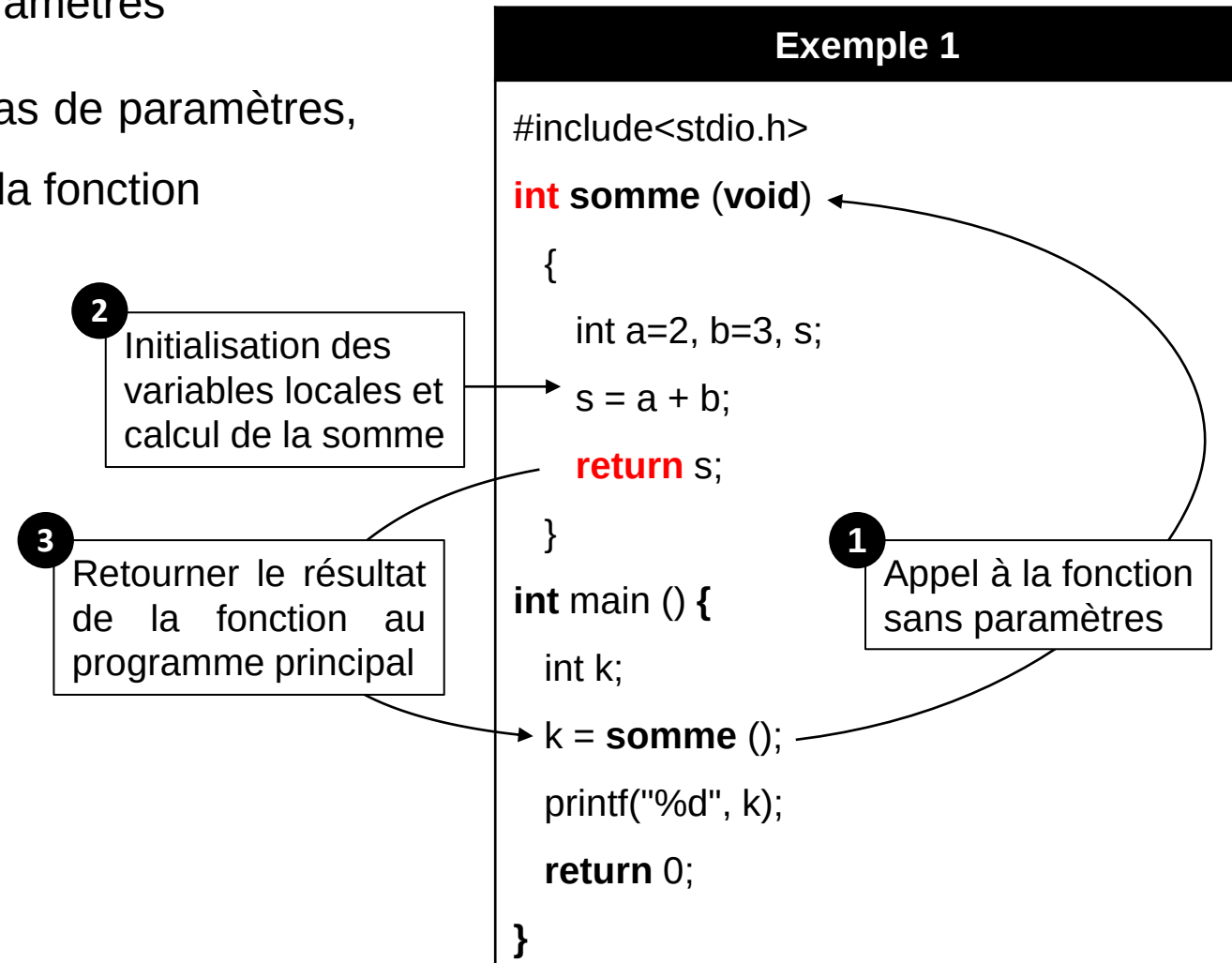
Retourner le résultat de la fonction au programme principal

3. Appels de fonction

- En C, une fonction peut ne pas avoir des paramètres
- Pour indiquer qu'une fonction ne possède pas de paramètres, on écrit le mot clé **void** dans la définition de la fonction

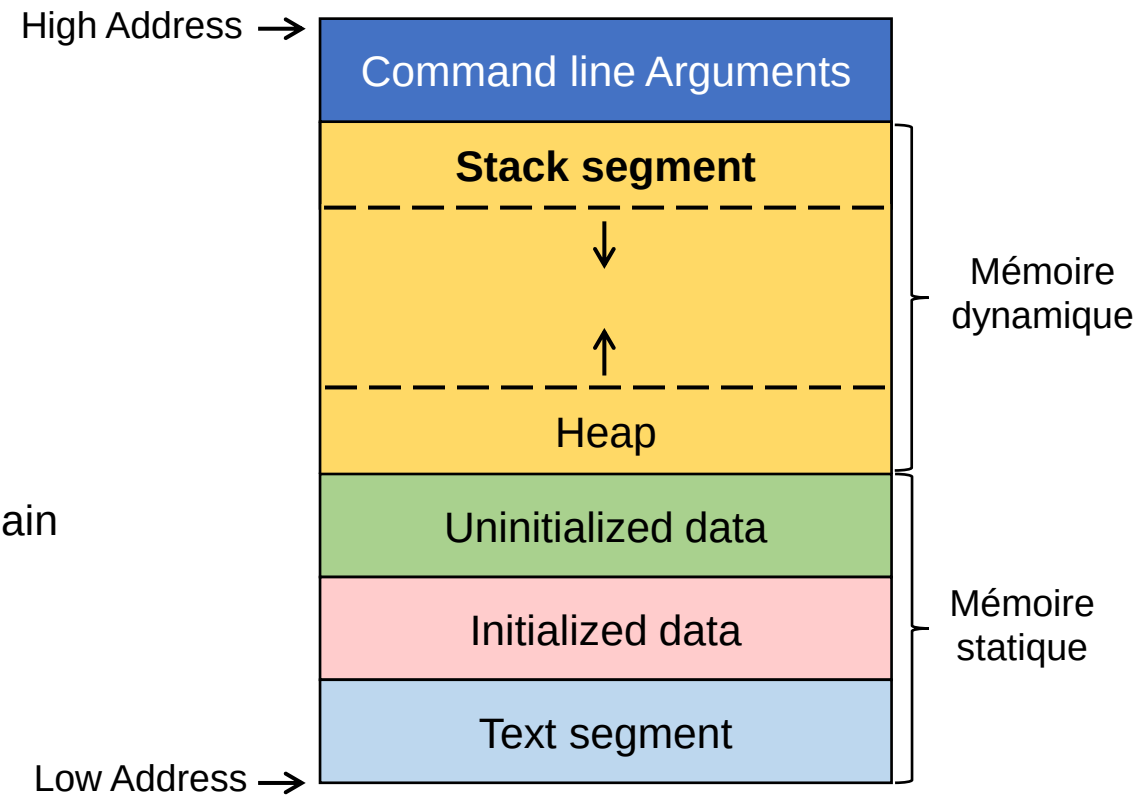
Inconvénient

- **Rigidité** La fonction exécute toujours la même tâche car elle ne peut pas être personnalisée avec des données d'entrée variables



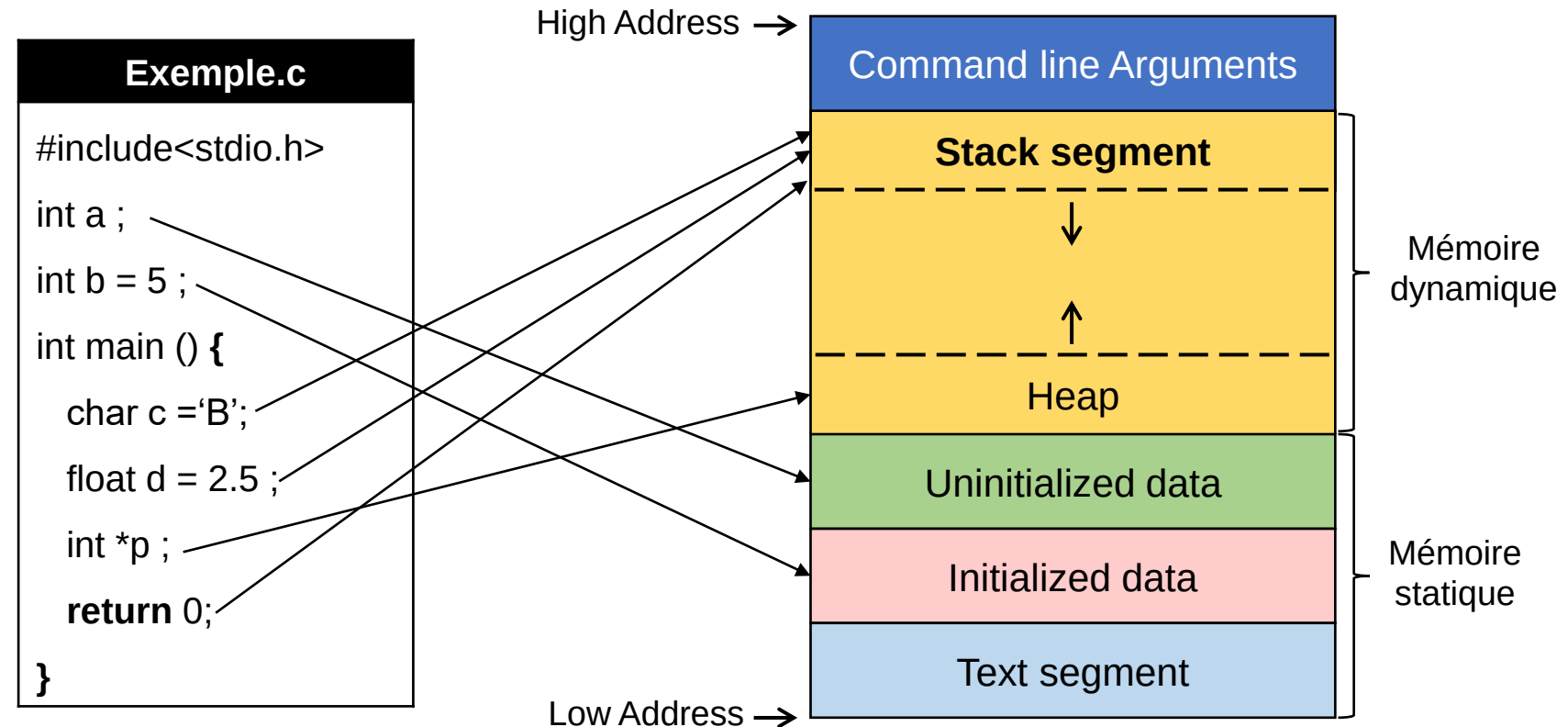
3. Appels de fonction

- Lorsqu'un programme C est exécuté, la fonction main est stockée dans une zone mémoire appelée **segment de pile (stack segment)**
- Dans ce segment de pile sont stockées:
 - Les variables locales de la fonction main
 - Les appels de fonction
 - Les paramètres des fonctions
 - Les variables locales des fonctions
 - Les adresses de retour des fonctions et de la fonction main



3. Appels de fonction

- Lorsqu'un programme C est exécuté, la fonction main est stockée dans une zone mémoire appelée **segment de pile (stack segment)**



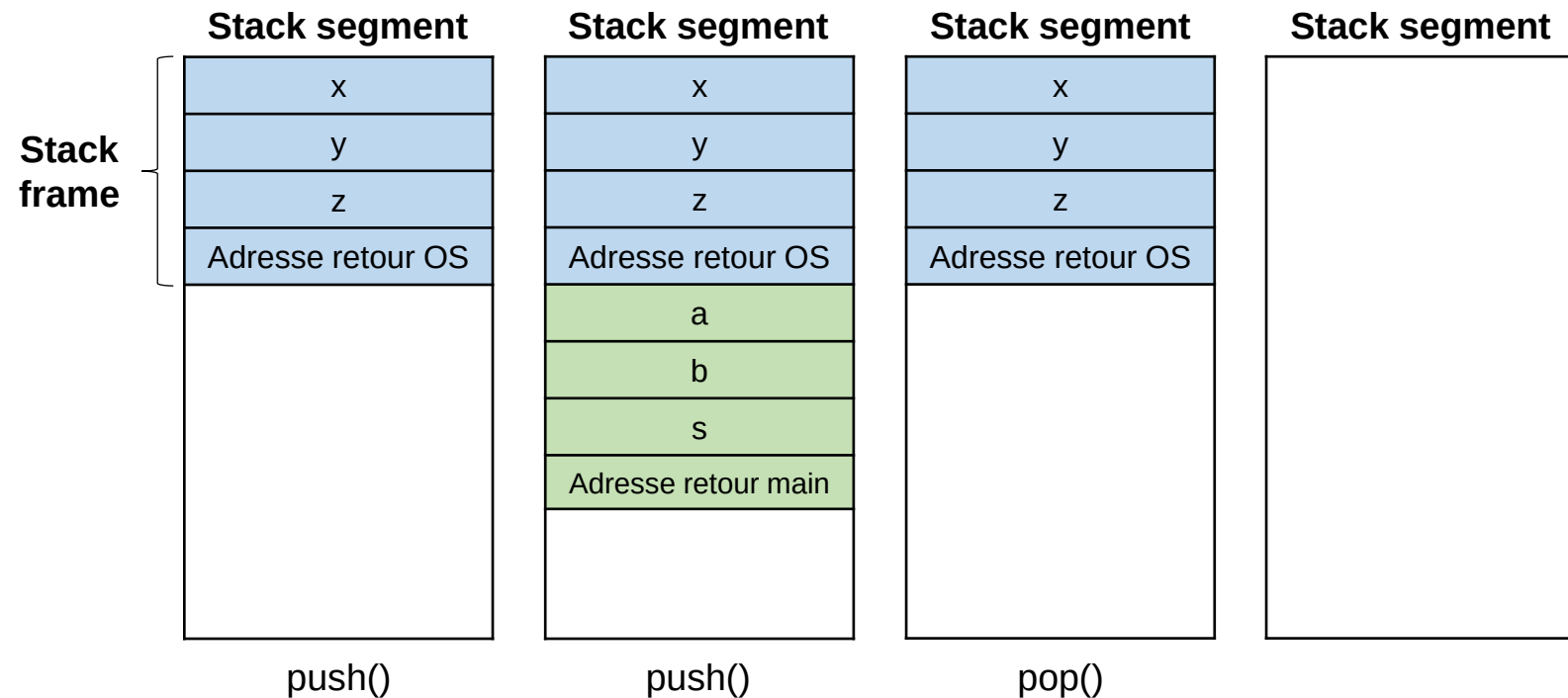
3. Appels de fonction

- La segment de pile fonctionne selon le principe de LIFO (Last In First Out)

Exemple.c

```
#include<stdio.h>
int somme (int a , int b)
{
    int s ;
    s = a + b ;
    return s ;
}

int main () {
    int x = 2, y = 3, z ;
    z = somme (x, y) ;
    return 0;
}
```



3. Appels de fonction

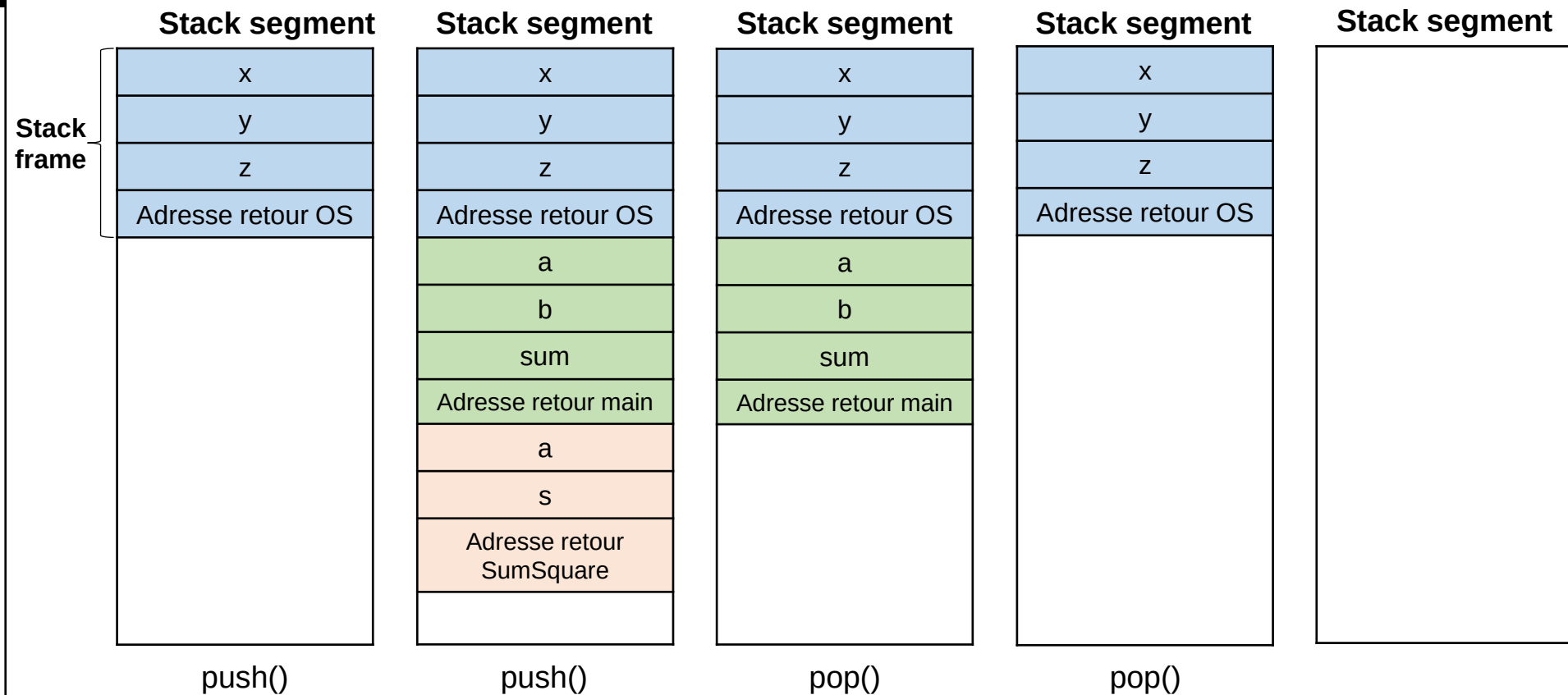
- La segment de pile fonctionne selon le principe de LIFO (Last In First Out)

Exemple.c

```
#include<stdio.h>
int square (int a) {
    int s ;
    s = a * a ;
    return s ;
}

int SumSquare (int a, int b) {
    int sum;
    sum = square (a+b);
    return sum;
}

int main () {
    int x = 2, y = 3, z ;
    z = SumSquare (x, y) ;
    return 0;
}
```



4. Passage des paramètres

- La communication entre le programme principale et les fonctions, ou entre les fonctions se fait à travers les **paramètres** des fonctions et l'instruction **return**
- Dans le langage C, on distingue entre deux modes pour transmettre des paramètres effectifs à une fonction:
 - **Passage par valeur** : la fonction reçoit en paramètre la valeur d'une donnée
 - **Passage par adresse** : la fonction reçoit en paramètre l'adresse d'une donnée

Rappel
<pre>#include<stdio.h> int main () { int a = 5 ; printf("La valeur de a est %d", a); printf("Adresse memoire de a est %p", &a); return 0; }</pre>

	Adresse	Valeur
a	0xA11BF	5

4.1. Passage par valeur

- Le passage par valeur est le mode standard de transmission de paramètres effectifs à une fonction
- La fonction appelée reçoit une **copie** de la valeur de l'objet passé comme paramètre effectif
- Ce type de passage permet de modifier le contenu de la copie locale de la fonction appelée et non pas la valeur de la variable originale

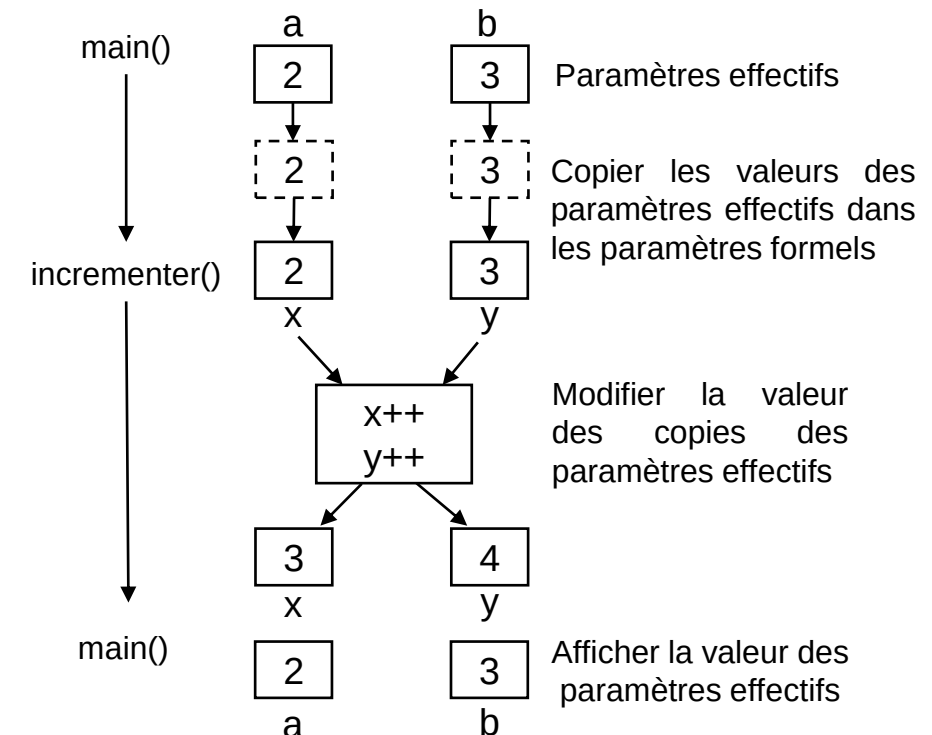
Exemple.c

```
#include<stdio.h>
int incrementer (int x, int y){
    x++;
    y++;
    return x+y;
}
int main(){
    int a=2, b=3, s;
    s = incrementer(a, b);
    printf("%d + %d = %d", a, b, s);
    return 0;
}
```



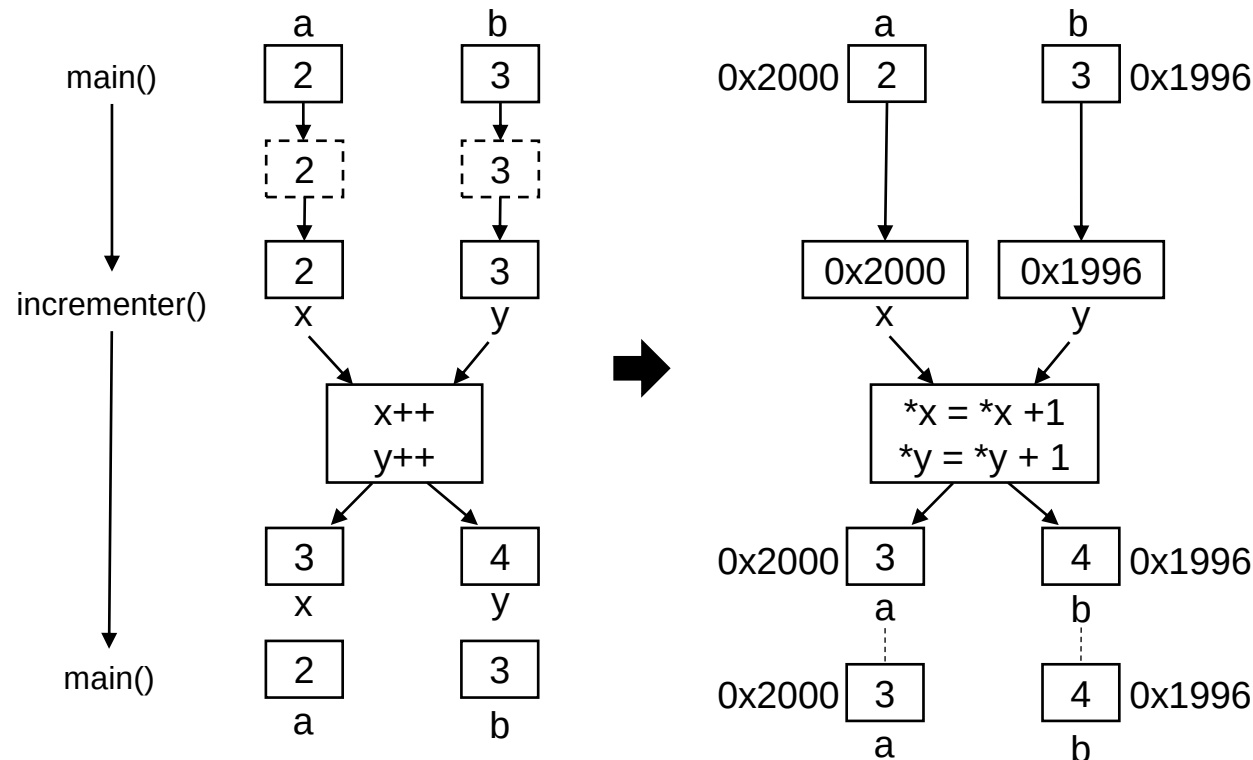
Résultat

2 + 3 = 7



4.2. Passage par adresse (par référence)

- Pour permettre à une fonction de modifier la valeur d'une donnée passée comme paramètre effectif, il faut lui transmettre l'adresse de l'objet concerné au lieu de sa valeur
- La fonction appelée range l'adresse envoyée dans le paramètre formel approprié, ce qui permet à la fonction de modifier les valeurs des paramètres effectifs originales



Exemple.c

```
#include<stdio.h>
int incrementer (int *a, int *b){
    *a = *a + 1;
    *b = *b + 1;
    return *a + *b;
}
int main(){
    int a=2, b=3, s;
    s = incrementer(&a, &b);
    printf("%d+%d=%d", a, b, s);
    return 0;
}
```

4.2. Passage par adresse

Enoncé

Ecrire en C un programme qui permet de permuter les valeurs de deux variables a et b.

Remarque: utiliser le passage par adresse lors de l'appel de la fonction permuter

Permuter.c

```
#include<stdio.h>
void permuter(int *x, int *y){
    int c;
    c = *x;
    *x = *y;
    *y = c;
}
int main(){
    int a=5, b=6;
    permuter(&a, &b);
    printf("a=%d et b=%d", a, b);
    return 0;
}
```